

Trabajo Práctico Integrador

El TP de la materia tiene como objetivo la integración productiva de los conceptos y herramientas vistas en la materia mediante el desarrollo de un proyecto grupal a elección como caso de aplicación.

Metodología

- El trabajo se desarrollará en grupos de 2 o 3 personas.
- Se deberá cumplir con 3 entregas y una demo en vivo.
- Como cierre del trabajo, deberán realizar una exposición oral en la última semana de cursada con posibilidad de recuperación en la semana de recuperatorios.
- El caso deberá ser propuesto a la cátedra y validado por la misma antes de dar por aprobada la entrega 1.
- Presentación en el próximo AI Fest o evento afín sujeto a disponibilidad.

Características requeridas del caso de aplicación

- Todo proyecto, sin excepción, debe tener:
 - Una arquitectura de sistema con un patrón definido y justificado (basada en servicios, orientada a eventos, por capas, etc.).
 - Persistencia de datos con un modelo de datos documentado.
 - Automatización de la integración y el despliegue.
 - Readiness para correr en infraestructura cloud.
- Debe presentar y cumplir con requisitos funcionales y no funcionales **razonables para un proyecto real en producción**.
- Debe incorporar de manera sustantiva múltiples de las siguientes áreas:
 - Ingeniería de datos: pipelines de ingesta y transformación sobre datos reales.
 - Calidad, trazabilidad y gobernanza de datos.
 - Combinación de motores de almacenamiento, cada uno respondiendo a una necesidad real del sistema.
 - Machine learning en producción: entrenamiento, puesta en producción y monitoreo.
 - Procesamiento orientado a eventos o streaming como mecanismo central de integración.
 - Observabilidad y operación del sistema.

Validación de proyecto

Aprobación por parte de la cátedra del plan de proyecto a desarrollar. La propuesta se presenta en un documento breve (1 a 2 carillas) que incluya:

- Problema a resolver y usuarios/actores del sistema.
- Solución propuesta a alto nivel: componentes principales, qué requisitos de base cubre y qué áreas del Caso incorpora.

- Fuentes de datos identificadas y verificadas, con una exploración inicial de los datos indicando por ejemplo volumen aproximado, frecuencia de actualización y licencia o condición de acceso.
- Viabilidad preliminar: por qué el proyecto es realizable por 2-3 personas en el tiempo de la cursada, cuál es el mayor riesgo técnico identificado, y qué plan B hay si ese riesgo o los datos fallan.
- Alcance tentativo del MVP: qué queda adentro y qué queda explícitamente afuera.

La cátedra puede aprobar, pedir ajustes de alcance (recortar o ampliar) o rechazar la propuesta. Si se piden ajustes, el grupo tiene una semana para volver a presentar. No se puede avanzar a la entrega de Diseño sin proyecto aprobado. Si después de la aprobación el grupo necesita cambiar el alcance o pivotear el proyecto, debe re-validarlo con la cátedra antes de seguir.

Entregas

Entrega 1: Diseño: PRD y Roadmap

- El PRD (Product Requirements Document) debe definir el producto desde la perspectiva del usuario: problema y contexto, usuarios y casos de uso, requisitos funcionales priorizados, requisitos no funcionales, criterios de éxito medibles, y supuestos y limitaciones.
- Fuentes de datos y viabilidad: profundización del análisis presentado en la aprobación, confirmando que los datos alcanzan para el alcance propuesto y que el proyecto es viable con los recursos y tiempos de la cursada. Los supuestos fuertes deben estar explicitados junto con un plan B.
- El Roadmap debe mapear el trabajo a las entregas restantes (POC y MVP): hitos y orden de construcción de la solución.

Entrega 2: POC y SRD

Prueba de concepto que valide el camino técnico de punta a punta, con el riesgo más grande del proyecto resuelto:

- El flujo principal del sistema debe funcionar de extremo a extremo con datos reales, aunque sea con alcance reducido (un subconjunto de datos, un solo caso de uso, interfaz mínima o inexistente).
- Las buenas prácticas de desarrollo colaborativo, integración continua y testing vistas en la materia deben estar operativas desde esta etapa.
- El sistema debe poder levantarse localmente de manera simple, con el procedimiento documentado en el README.
- ADRs de las decisiones estructurales tomadas hasta acá.
- Está permitido y esperado que haya partes mockeadas o simplificadas; deben estar identificadas en el README.
- El SRD (System Requirements/Design Document) debe traducir el PRD a una especificación técnica: arquitectura del sistema con su patrón y diagrama, stack tecnológico justificado, modelo de datos, contratos de las interfaces principales y estrategia de datos. Sin incluir aún los componentes de ML, el enfoque de entrenamiento, serving y monitoreo.

Entrega 3: MVP

Versión mínima pero completa y operable del producto definido en el PRD:

- Todos los requisitos funcionales priorizados como imprescindibles en el PRD implementados y funcionando; los recortes respecto del diseño original documentados y justificados.
- El ciclo de vida completo del software automatizado, y el sistema listo para correr en infraestructura cloud (readiness demostrable aunque el servicio no quede permanentemente hosteado).
- Los requisitos no funcionales declarados en el PRD cumplidos de manera verificable, incluyendo la capacidad de observar y operar el sistema.
- Pipeline reproducible para el entrenamiento de modelos de ML, incluyendo el proceso definido para la actualización y puesta en producción (serving) del modelo.
- El sistema completo debe poder levantarse localmente con el procedimiento documentado: es el mecanismo principal de corrección, independiente de que haya o no un deploy vivo.
- ADRs actualizados cubriendo las decisiones clave de esta etapa.
- SRD completo incluyendo los componentes de ML, el enfoque de entrenamiento, serving y monitoreo.

Común a todas las entregas

- Con cada entrega deben incluir un video de entre 5 y 12 minutos explicando el desarrollo y el razonamiento detrás. En el video debe demostrarse funcionalmente lo desarrollado del sistema.
- En cada entrega subsiguiente, deben verse subsanadas las correcciones e incluidas las consideraciones provistas por la cátedra.

Fechas de Entrega

- Entrega 1: Diseño (PRD y Roadmap): a la mitad de la cursada durante la semana de parciales, previa aprobación del proyecto.
- Entrega 2: El POC se espera al terminar la unidad 2.
- Entrega 3: El MVP se entrega la última semana de clases.

Formato de Entrega

La entrega se realizará a través del campus virtual, con un link al commit específico y URL del video hosteado con acceso público.

Cada entrega estará compuesta por los siguientes ítems:

- Commit al repositorio de git que se tomará para la entrega. El commit debe estar en la rama principal e informarse explícitamente por el canal de entrega; lo que no esté integrado a la rama principal al momento de la entrega no se corrige. En este commit deberíamos encontrar:
 - README actualizado con instrucciones para acceder a todos los componentes del desarrollo.

- Directorio con ADRs ([Architecture Decision Record](#)) para las decisiones de diseño tomadas en cada entrega (este tema será expandido en la materia).
 - El objetivo de estos archivos es explicitar los análisis realizados durante el desarrollo, lo cual es un concepto de evaluación como se menciona luego en este documento.
 - Sugerimos utilizar un formato simple como [este](#).
 - Los ADRs que no lleven a cabo una comparación de alternativas, o que únicamente describan el camino tomado, serán considerados inválidos a los fines de la evaluación.
- Video de entre 5 y 12 minutos a modo de demo del desarrollo realizado.

Criterios de evaluación

Cada entrega se evalúa sobre las siguientes dimensiones, en función de todo lo visto en la materia. La vara general es: lo que el equipo declara (en README, ADRs o video) debe ser verificable en el repositorio.

- Alcance y complejidad. Sistema de múltiples componentes, con una complejidad justificada por el problema y realista respecto del alcance aprobado.
- Arquitectura y diseño. Coherencia entre el patrón declarado y lo efectivamente construido, con las decisiones estructurales registradas en ADRs.
- Prácticas de desarrollo. Buenas prácticas de desarrollo colaborativo, testing y automatización, con una historia de trabajo que refleje el desarrollo real y la contribución de cada integrante del equipo, sin excepción.
- Ingeniería y calidad de datos. Modelo de datos coherente con el uso, procesos de datos confiables y repetibles, y calidad de los datos atendida como parte del diseño.
- Operabilidad e infraestructura. Facilidad para levantar, desplegar, observar y diagnosticar el sistema.
- Robustez, seguridad y escalabilidad. Manejo de errores, control de acceso en las interfaces expuestas, manejo responsable de credenciales, y un diseño que identifica sus cuellos de botella y explica cómo escalaría, sin necesidad de implementar esa escala.
- Validez interna y externa. Interna: consistencia entre lo declarado y lo construido; el PRD, el SRD, los ADRs y el código cuentan la misma historia. Externa: el sistema resuelve un problema plausible con datos reales; los resultados que muestra son honestos y verificables, no salidas fabricadas para la demo.
- Defensa. Capacidad del equipo para responder preguntas sobre el diseño y funcionamiento del sistema: supuestos y limitaciones, trade-offs de las alternativas analizadas y justificación técnica de las soluciones desarrolladas. Aplica a los videos y a la exposición oral, y es individual además de grupal: cada integrante debe poder explicar el sistema completo.

Para cada entrega devolveremos un APROBADO, REVISAR o DESAPROBADO, junto con correcciones a realizar. La nota final del trabajo será numérica, a consideración de la cátedra en base a todos los entregables.

Comunicación durante el desarrollo

- Usaremos un foro en Discord, con hilos por grupo, para despejar dudas.

Consideraciones

- La originalidad suma pero NO es un requisito. El objetivo es que simule un proyecto real de software en producción.
- El repositorio debe ser privado y compartido con el equipo docente desde el inicio del desarrollo.
- Datos: usar datasets públicos, propios o sintéticos. Si los datos son sintéticos, el mecanismo de generación es parte del sistema y se evalúa.
- Costos de infraestructura: el proyecto debe poder desarrollarse y corregirse sin costos significativos: free tiers, recursos efímeros o ejecución local. Ningún criterio de evaluación exige mantener servicios cloud pagos encendidos fuera de las demos.
- Videos: subidos a una plataforma accesible (link en el README o en el canal de entrega), disponibles durante todo el período de corrección. El video es evidencia funcional: lo que no se pueda verificar por código y no esté demostrado en el video se considera no hecho.
- El uso de asistentes de IA está permitido; la evaluación es sobre el criterio de ingeniería: en la defensa y los videos el equipo debe poder explicar, justificar cada parte del sistema como propia y razonar sobre contrafacticos planteados.